

Learning the Score Function with Neural Nets

May 21, 2026

- Goal: Learning the Score with Neural Nets
- The Problem with the Normal Loss Function
- The Fix: Denoising Score Matching (DSM)
- Why the Fix Works (Equivalence)
- Fixing Exploding Gradients (Time Weighting)
- The Training Loop

Recap: Forward and Reverse Diffusion

- **Forward process:** Adding noise over time: $dx = -\frac{1}{2}\beta x dt + \sqrt{\beta} dW$
- **Transitions:** Jumping straight to time t : $x_t = \alpha_t x_0 + \sigma_t \epsilon$
- **Parameters:** $\epsilon \sim \mathcal{N}(0, I)$, $\alpha_t = e^{-\frac{1}{2}\beta t}$, $\sigma_t = \sqrt{1 - e^{-\beta t}}$
- **Reverse process:** Generating data by running time backwards:
 $dx = \left[\frac{1}{2}\beta x - \beta \nabla_x \log p_t(x) \right] dt + \sqrt{\beta} d\bar{W}$
- **Requirement:** We need the unknown score $\nabla_x \log p_t(x)$ to do this

Goal: Learning the Score with Neural Nets

- **Goal:** Learn the real score $\nabla_{x_t} \log p_t(x_t)$
- **Model:** A neural net predicting the score: $s_\theta(x_t, t)$
- **Optimization:** Tune the weights θ to make the error as small as possible

The Problem with the Normal Loss Function

- **Standard loss:** Just normal mean squared error against the true score

$$L(\theta) = \mathbb{E}_{t \sim \mathcal{U}(0, T)} \mathbb{E}_{x \sim p_t(x)} [\|\nabla_x \log p_t(x) - s_\theta(x, t)\|_2^2]$$

- **Batch estimation:** Averaging the error over random samples

$$L(\theta) \approx \frac{1}{N} \sum_{i=1}^N \|\nabla_x \log p_{t_i}(x_i) - s_\theta(x_i, t_i)\|_2^2$$

- **Blocker:** We don't actually know the true score $\nabla_x \log p_t(x)$!

The Fix: Denoising Score Matching (DSM)

- **Solution:** Use the conditional score $\nabla_{x_t} \log p_t(x_t|x_0)$ starting from a known real data point x_0
- **Forward distribution:** It's just a simple Gaussian curve

$$p_t(x_t|x_0) = \mathcal{N}(x_t; \alpha_t x_0, \sigma_t^2 I)$$

- **Analytical gradient:** Taking the derivative is easy

$$\nabla_{x_t} \log p_t(x_t|x_0) = \nabla_{x_t} \left(-\frac{\|x_t - \alpha_t x_0\|^2}{2\sigma_t^2} \right) = -\frac{x_t - \alpha_t x_0}{\sigma_t^2}$$

- **Simplification:** Plugging in our noisy data formula $x_t = \alpha_t x_0 + \sigma_t \epsilon$ leaves just the noise

$$\nabla_{x_t} \log p_t(x_t|x_0) = -\frac{\epsilon}{\sigma_t}$$

Why the Fix Works (Equivalence)

- **Theorem:** Minimizing the easy conditional loss gives the exact same result as the impossible normal loss!

$$L_C(\theta) = \mathbb{E}_{t, x_0, x_t} \left[\|\nabla_{x_t} \log p_t(x_t | x_0) - s_\theta(x_t, t)\|_2^2 \right]$$

- **Takeaway:** We can train the network perfectly just by adding random noise to real data

Fixing Exploding Gradients (Time Weighting)

- **Problem:** Near $t = 0$, the noise size gets super small ($\sigma_t \rightarrow 0$)
- **Consequence:** Dividing by tiny numbers makes the gradients explode!
- **Fix:** Multiply the loss by a weight factor $\lambda(t) = \sigma_t^2$
- **Result:** Stable training just predicting the pure noise

$$\mathcal{L}(\theta) = \mathbb{E}_{t, x_0, \epsilon} \left[\sigma_t^2 \left\| s_\theta(x_t, t) + \frac{\epsilon}{\sigma_t} \right\|_2^2 \right]$$

- **Validity:** Multiplying by a positive number changes nothing about where the best solution is

The Training Loop

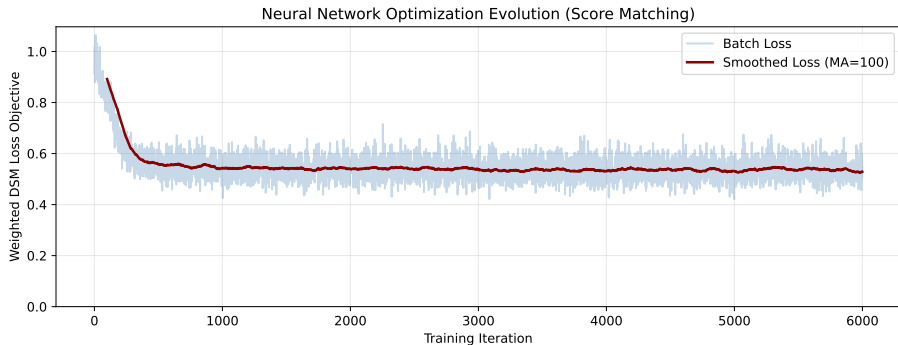
- 1 Pick random times $t \sim \mathcal{U}(0, T)$ and real starting data x_0
- 2 Calculate time constants $\alpha_t = e^{-\frac{1}{2}\beta t}$ and $\sigma_t = \sqrt{1 - e^{-\beta t}}$
- 3 Generate noise $\epsilon \sim \mathcal{N}(0, I)$ to create noisy versions $x_t = \alpha_t x_0 + \sigma_t \epsilon$
- 4 Get the neural net's prediction $s_\theta(x_t, t)$
- 5 Calculate the weighted error:

$$\mathcal{L}(\theta) = \sigma_t^2 \left\| s_\theta(x_t, t) + \frac{\epsilon}{\sigma_t} \right\|_2^2$$

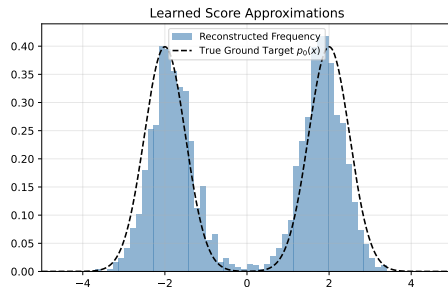
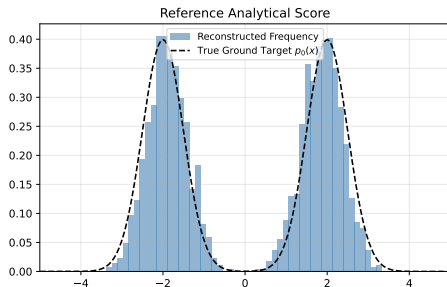
- 6 Update weights using gradient descent:

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}(\theta)$$

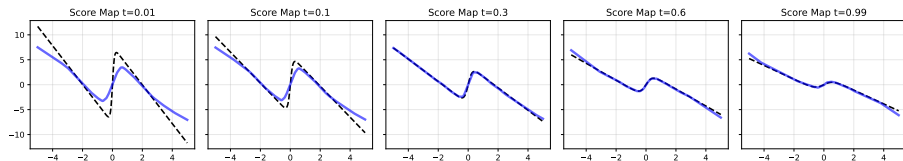
Results: Training Convergence



Results: True vs Learned Stationary Distribution

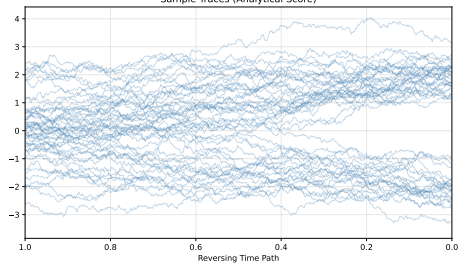


Results: Score Function Vector Field

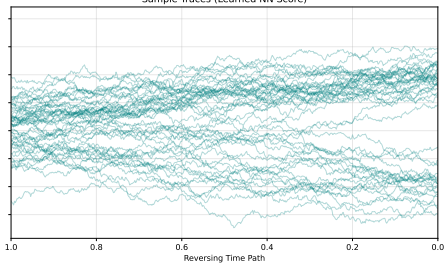


Results: Backward Path Generation

Sample Traces (Analytical Score)



Sample Traces (Learned NN Score)



Results: Probability Evolution Over Time

